Digital Image Analysis Tutorial 12

```python
#!/usr/bin/env python3
"""
AI531C — T12: The Loss Ablation Studio

Run:
    python3 T12_studio.py
    python3 T12_studio.py support
    python3 T12_studio.py extension
    python3 T12_studio.py --image path/to/image.png

Install:
    pip install numpy scipy scikit-image
"""

import argparse
import numpy as np
from scipy.ndimage import gaussian_filter
from skimage import data, io, color, img_as_float32
from skimage.metrics import (
    mean_squared_error, peak_signal_noise_ratio, structural_similarity
)

PATCH = 5
SEED = 12
SIGMA = 15


def load_image(path=None):
    image = io.imread(path) if path else data.camera()
    if image.ndim == 3:
        if image.shape[-1] == 4:
            image = image[..., :3]
        image = color.rgb2gray(image)
    image = img_as_float32(image)
    if image.ndim != 2 or min(image.shape) < PATCH + 2:
        raise ValueError("Provide a sufficiently large grayscale or RGB image.")
    return np.clip(image, 0, 1)
```

```python
def add_noise(image, sigma, rng):
    return np.clip(image + rng.normal(0, sigma / 255.0, image.shape), 0, 1)


def make_patches(clean, count, rng):
    noisy = add_noise(clean, SIGMA, rng)
    radius = PATCH // 2
    rows = rng.integers(radius, clean.shape[0] - radius, count)
    cols = rng.integers(radius, clean.shape[1] - radius, count)
    x = np.empty((count, PATCH * PATCH), dtype=np.float64)
    y = clean[rows, cols].astype(np.float64)
    for i, (r, c) in enumerate(zip(rows, cols)):
        x[i] = noisy[r-radius:r+radius+1, c-radius:c+radius+1].ravel()
    return x, y


def train_model(clean, count, seed=SEED):
    rng = np.random.default_rng(seed)
    x, y = make_patches(clean, count, rng)
    xm, ym = x.mean(axis=0), y.mean()
    xc, yc = x - xm, y - ym
    weights = np.linalg.solve(
        xc.T @ xc + 1e-5 * np.eye(x.shape[1]), xc.T @ yc
    )
    bias = ym - xm @ weights
    return weights, bias


def denoise(noisy, model):
    radius = PATCH // 2
    windows = np.lib.stride_tricks.sliding_window_view(noisy, (PATCH, PATCH))
    features = windows.reshape(-1, PATCH * PATCH)
    weights, bias = model
    return np.clip(features @ weights + bias, 0, 1).reshape(
        noisy.shape[0] - 2 * radius, noisy.shape[1] - 2 * radius
    )
```

```python
def psnr(reference, estimate):
    return float(peak_signal_noise_ratio(reference, estimate, data_range=1.0))


def run_judges(clean):
    print("\\nPART 1 — THREE JUDGES")
    print("T12 sheet reference values (MAE was not specified on the sheet):")
    print("-" * 65)
    print(f"{'Candidate':<25}{'MSE (8-bit²)':>15}{'SSIM':>12}")
    print(f"{'Blur':<25}{189:>15}{0.733:>12.3f}")
    print(f"{'Brightness offset +16':<25}{255:>15}{0.951:>12.3f}")
    print("MSE prefers blur (189 < 255), while SSIM prefers the offset image.")
    print("\\nMetrics recalculated on the selected image:")
    print(f"{'Candidate':<25}{'MSE (8-bit²)':>15}{'MAE (8-bit)':>15}{'SSIM':>10}")
    candidates = [
        ("Blur", gaussian_filter(clean, sigma=1.5)),
        ("Brightness offset +16", np.clip(clean + 16 / 255.0, 0, 1)),
    ]
    for name, candidate in candidates:
        mse = mean_squared_error(clean, candidate) * 255**2
        # MAE is computed with NumPy; scikit-image does not expose this metric.
        mae = np.mean(np.abs(clean - candidate)) * 255
        ssim = structural_similarity(clean, candidate, data_range=1.0)
        print(f"{name:<25}{mse:>15.2f}{mae:>15.2f}{ssim:>10.3f}")


def run_ablation(clean):
    print("\\nPART 2 — PATCH-COUNT ABLATION")
    print("Only patch count changes. Training seed and test noise are fixed.")
    test_noisy = add_noise(clean, SIGMA, np.random.default_rng(SEED + 1000))
    radius = PATCH // 2
    reference = clean[radius:-radius, radius:-radius]
    results = {}
    for count in (5000, 25000):
        model = train_model(clean, count, SEED)
        output = denoise(test_noisy, model)
        results[count] = psnr(reference, output)
    print("-" * 50)
```

```
104  def run_ablation(clean):
116      print(f"{'Training patches':<24}{'PSNR (dB)':>14}")
117      for count, score in results.items():
118          print(f"{count:<24,}{score:>14.2f}")
119      print(f"Absolute difference: {abs(results[25000] - results[5000]):.4f} dB")
120      print("T12 reference run: 5k = 29.45 dB; 25k = 29.48 dB (0.03 dB).")
121      print("A tiny change is not automatically proof of no effect: check run-to-run")
122      print("variation and whether enough repetitions/samples were used.")
123      return results
124
125
126  def run_extension(clean):
127      blur = gaussian_filter(clean, sigma=1.5)
128      offset = np.clip(clean + 16 / 255.0, 0, 1)
129      candidates = {"Blur": blur, "Brightness offset +16": offset}
130      scores = {}
131      for name, image in candidates.items():
132          scores[name] = (
133              mean_squared_error(clean, image),
134              structural_similarity(clean, image, data_range=1.0)
135          )
136      max_mse = max(v[0] for v in scores.values())
137      print("\\nPART 3 — MIX-LOSS SWEEP")
138      print("Assumed loss = (1-lambda)*(MSE/max_MSE) + lambda*(1-SSIM)")
139      print("Lower loss wins. The tutorial sheet does not specify a mix-loss formula.")
140      print("-" * 68)
141      print(f"{'Lambda':<12}{'Preferred':<25}{'Blur loss':>14}{'Offset loss':>14}")
142      for lam in (0.0, 0.2, 0.5):
143          losses = {
144              name: (1-lam) * mse / max_mse + lam * (1-ssim)
145              for name, (mse, ssim) in scores.items()
146          }
147          winner = min(losses, key=losses.get)
148          print(f"{lam:<12.1f}{winner:<25}{losses['Blur']:>14.4f}{losses['Brightness
             offset +16']:>14.4f}")
```

```python
def protocol_table():
    print("\\nSIX-ROW EVALUATION PROTOCOL — ENTER YOUR MODEL'S RESULTS")
    rows = [
        ("1", "Clean reference", "No corruption", "MSE / MAE / SSIM"),
        ("2", "Blurred input", "Gaussian blur", "MSE / MAE / SSIM"),
        ("3", "Brightness offset", "+16 intensity", "MSE / MAE / SSIM"),
        ("4", "Matched noise", "Train/test sigma = 15", "PSNR / SSIM"),
        ("5", "Noise mismatch", "Train 15, test 30", "PSNR / SSIM"),
        ("6", "Patch ablation", "5k vs 25k; fixed seed", "PSNR difference"),
    ]
    print("-" * 100)
    print(f"{'Row':<6}{'Test':<23}{'Condition':<34}{'Metrics':<24}{'Your result'}")
    for row in rows:
        print(f"{row[0]:<6}{row[1]:<23}{row[2]:<34}{row[3]:<24}________")


def discussion():
    print("\\nDISCUSSION")
    print("Q1. Radiology: preserving diagnostically important structure is critical;")
    print("    pixel error alone cannot establish clinical usefulness or safety.")
    print("    Photo apps: the right metric depends on the goal—pixel fidelity,")
    print("    perceptual similarity, or preservation of brightness and colour.")
    print("Q2. Check run-to-run variability and use enough repetitions/samples to")
    print("    decide whether the measured change is distinguishable from noise.")
    print("Q3. Different seeds change sampled patches and noise; this variation can")
    print("    masquerade as the effect of the variable being ablated.")
```

```python
def main():
    parser = argparse.ArgumentParser(description="AI531C T12 loss ablation studio")
    parser.add_argument("mode", nargs="?", choices=["support", "extension"])
    parser.add_argument("--image", default=None, help="Optional grayscale/RGB image
    path")
    args = parser.parse_args()

    clean = load_image(args.image)
    print("AI531C — T12: The Loss Ablation Studio")
    print(f"Image: {args.image or 'skimage.data.camera()'}")
    print(f"Image size: {clean.shape[1]} x {clean.shape[0]}")

    run_judges(clean)
    if args.mode == "support":
        print("\\nSUPPORT MODE — judges' table only.")
        print("Medical-danger question: a metric may reward low pixel error while")
        print("important anatomical structure is blurred or removed. A metric score")
        print("alone must never establish that an image is clinically safe.")
        discussion()
        return

    if args.mode == "extension":
        run_extension(clean)
        protocol_table()
        discussion()
        return

    run_ablation(clean)
    protocol_table()
    discussion()
    print("\\nUse --image path/to/image.png to test your own image.")


if __name__ == "__main__":
    main()
```

Output:

```
$ python T12_studio.py
AI531C — T12: The Loss Ablation Studio
Image: skimage.data.camera()
Image size: 512 x 512
\nPART 1 — THREE JUDGES
T12 sheet reference values (MAE was not specified on the sheet):
------------------------------------------------------------------
Candidate                   MSE (8-bit²)        SSIM
Blur                                 189       0.733
Brightness offset +16                255       0.951
MSE prefers blur (189 < 255), while SSIM prefers the offset image.
\nMetrics recalculated on the selected image:
Candidate                   MSE (8-bit²)    MAE (8-bit)      SSIM
Blur                              120.35           5.73     0.803
Brightness offset +16             254.90          15.94     0.951
\nPART 2 — PATCH-COUNT ABLATION
Only patch count changes. Training seed and test noise are fixed.
--------------------------------------------------
Training patches             PSNR (dB)
5,000                            29.47
25,000                           29.49
Absolute difference: 0.0233 dB
T12 reference run: 5k = 29.45 dB; 25k = 29.48 dB (0.03 dB).
A tiny change is not automatically proof of no effect: check run-to-run
variation and whether enough repetitions/samples were used.
\nSIX-ROW EVALUATION PROTOCOL — ENTER YOUR MODEL'S RESULTS
-------------------------------------------------------------------------------------------------
------
Row   Test                  Condition                       Metrics                 Your re
sult
1     Clean reference       No corruption                   MSE / MAE / SSIM        _______
_
2     Blurred input         Gaussian blur                   MSE / MAE / SSIM        _______
_
3     Brightness offset     +16 intensity                   MSE / MAE / SSIM        _______
_
4     Matched noise         Train/test sigma = 15           PSNR / SSIM             _______
_
5     Noise mismatch        Train 15, test 30               PSNR / SSIM             _______
_
6     Patch ablation        5k vs 25k; fixed seed           PSNR difference         _______
_
\nDISCUSSION
Q1. Radiology: preserving diagnostically important structure is critical;
    pixel error alone cannot establish clinical usefulness or safety.
    Photo apps: the right metric depends on the goal—pixel fidelity,
    perceptual similarity, or preservation of brightness and colour.
Q2. Check run-to-run variability and use enough repetitions/samples to
    decide whether the measured change is distinguishable from noise.
Q3. Different seeds change sampled patches and noise; this variation can
    masquerade as the effect of the variable being ablated.
\nUse --image path/to/image.png to test your own image.
```